Loyalty Programs, Loyalty Cards, and Loyalty Points

Application review for the current implementation in ereckontamil. Generated on 2026-07-16.

Short answer: the application has the three loyalty master/transaction models, but only Loyalty Cards and Loyalty Points are partially connected to POS flow today. Loyalty Program exists as configuration data, but current sale logic does not use it.

1. Existing Behavior

Loyalty Programs

Loyalty Programs store the intended business rules for the loyalty scheme. The form and migration show these fields:

- `name`
- `code`
- `points_per_amount`
- `minimum_points_to_redeem`
- `value_per_point`
- `status`

Current role: administrative setup only. There is no active code path in POS sale processing that reads these values when awarding or redeeming points.

Loyalty Cards

Loyalty Cards are customer-linked identities used to find the customer in billing.

- Each card belongs to one customer.
- Card fields include `card_number`, `issue_date`, `expiry_date`, and `status`.
- POS customer search checks card number as one of the search keys.

Current role: identification and lookup. The system uses the card to locate a customer record, not to drive rule validation or separate balance management.

Loyalty Points

Loyalty Points act as the transaction ledger for points earned and redeemed.

- Each row belongs to one customer.
- Transaction fields include `transaction_type`, `points`, `description`, and `transaction_date`.
- When a POS sale is created, the application inserts loyalty point rows for `earn` and `redeem` activity.

Current role: audit trail for the running `customers.total_points` balance.

2. How It Works Today in POS

Step	Current behavior
Customer search	Cashier can search by customer name, phone, or loyalty card number.
Customer snapshot	POS screen shows customer group, active loyalty card, available points, wallet balance, and GST.
Discount application	Customer group discount percentage is applied on item pricing during sale entry.
Point redemption	Cashier can manually enter a redemption amount on the sale.
Point earning	After sale creation, points are earned using a hardcoded formula: `(int) (net_amount / 100)`.
Point ledger update	The system inserts `loyalty_points` records for earn and redeem transactions.
Customer balance update	The system updates `customers.total_points` directly after the sale is saved.

> **Key implementation fact: Loyalty Program values are not used by the live POS earn/redeem logic. The current logic is hardcoded at sale creation time.**

3. What Each Model Is Supposed To Do

Model	Business purpose	Current state
LoyaltyProgram	Define how many points are earned, the minimum redeem threshold, and the value of	Configured in admin but not enforced in sale logic.

Model	Business purpose	Current state
	each point.	
LoyaltyCard	Give the customer a unique membership identity for lookup and membership verification.	Used for customer search; no rule enforcement based on card validity.
LoyaltyPoint	Store point earn/redeem history as a ledger.	Actively written from POS sale creation.

4. Gaps In The Current Implementation

- **Loyalty Program is not driving the real calculations.** Current earn rate ignores `points_per_amount`, `minimum_points_to_redeem`, and `value_per_point`.
- **No visible validation prevents over-redemption.** A cashier can enter a redemption amount without clear enforcement against the actual available balance.
- **No card validity check is enforced at redemption time.** Expired or inactive cards are not clearly blocked from use in the POS loyalty path.
- Point conversion is treated as amount-equals-points in the current sale hook, which is usually too simplistic for production use.
- No clear reversal flow is visible here for cancelled sales, edited sales, or returns affecting already posted loyalty entries.
- No direct linkage is stored from a loyalty point row to the source sale ID, only a description string is inserted.

5. What Should Be Done

Required for a production-safe loyalty module

1. Make POS sale logic read the active Loyalty Program before calculating earn or redeem values.
2. Use `points_per_amount` for earning, instead of the hardcoded `net_amount / 100` rule.
3. Use `minimum_points_to_redeem` to block redemption below the required threshold.
4. Use `value_per_point` to convert points into currency correctly.
5. Validate that entered redemption never exceeds the customer's available points.
6. Validate that the customer's loyalty card is active and not expired when card-based loyalty is required.

7. Store a direct source reference on loyalty transactions, such as POS sale ID, return ID, or manual adjustment ID.
8. Add reversal logic for sale cancellation, POS return, and edit scenarios.

Recommended operational improvements

1. Add a manual loyalty adjustment screen for admin-approved credit/debit corrections.
2. Add a customer loyalty statement that shows opening balance, transactions, and closing balance.
3. Add branch-level and date-range loyalty reports.
4. Prevent negative customer point balances at both form level and database transaction level.
5. Consider whether only one active loyalty program should exist at a time, and enforce that rule if needed.

6. Recommended Business Flow

Stage	Recommended behavior
Program setup	Admin creates one active loyalty program with earning rate, redemption threshold, and point value.
Card issue	Customer receives a loyalty card linked to the customer master record.
Sale billing	Cashier selects customer, system validates card status, then calculates discounts and eligible loyalty usage.
Redemption	System converts points to currency using the active loyalty program and blocks excess usage.
Posting	After successful sale, system updates customer point balance and writes the loyalty ledger with source references.
Reversal	If the sale is voided or returned, system posts reversing loyalty entries automatically.

7. Bottom Line

If you use the module exactly as it exists now, Loyalty Cards help identify the customer, Loyalty Points record earn/redeem movements, and Loyalty Programs mostly act as unused setup data. To

make the feature complete, the program rules must be connected to the POS posting logic and guarded with validation and reversal handling.